Módulo 1

Exemplo 1 (0 ponto)

Neste exemplo queremos fazer uma soma saturada entre dois valores: se a soma exceder o valor 0xFFFF, o resultado deve ser 0xFFFF.

A Figura 1 mostra um fluxograma para a solução deste exemplo. Neste caso, **o resultado ficará salvo no registrador R4**. Note que o início do fluxograma representa a inicialização de R4 e R5 com valores quaisquer (representados por X e Y) que são as entradas do programa.

Para verificar a condição de "haver carry", podemos usar a instrução JNC, que realiza um salto se a flag de carry estiver em 0. Note que **o valor da flag de carry estará de acordo com o resultado da soma R4 + R5**.

O fim do programa é um loop infinito (para manter o processador funcionando) que implementamos com JMP $.

A Listagem 1 mostra um código em assembly correspondente ao fluxograma da Figura 1. Crie um projeto, copie e cole este código em um arquivo com extensão ".asm" e faça testes para verificar que ele resolve o exemplo.

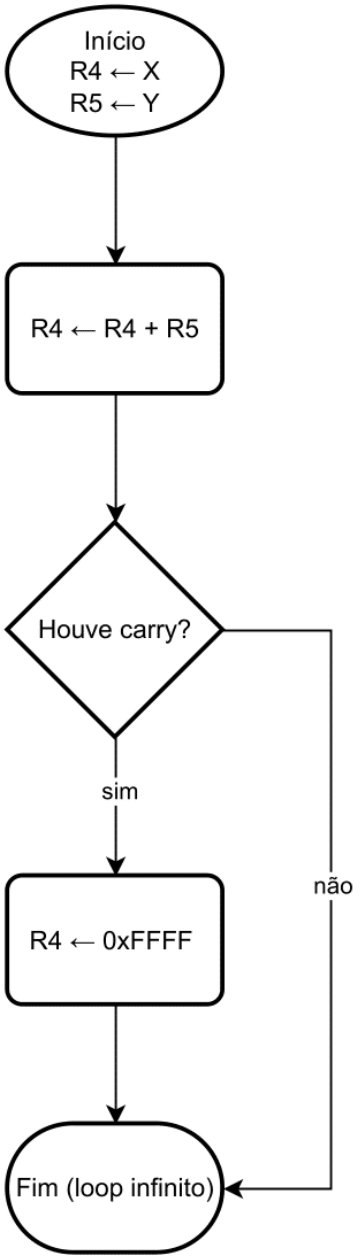


Figura 1 - Fluxograma para o Exemplo 1

Listagem 1 - Solução do Exemplo 1

```
1  ; Todos os nossos programas em assembly se iniciam com as linhas 2 a 8 desta listagem
2      .cdecls "msp430.h"
3      .global main
4      .text
5
6  main:
7      mov.w   #(WDTPW|WDTHOLD), &WDTCTL
8
9      ; Inicializar R4 e R5 com números quaisquer (teste com outros)
10     mov.w   #123, R4
11     mov.w   #456, R5
12
13     add.w   R5, R4 ; Fazer a soma: R4 = R4 + R5
14
15     jnc     FIM    ; Se não houve carry, pular para o rótulo "FIM"
16
17     ; A linha 18 só será executada se o salto não tiver acontecido, i.e., se houve carry
18     mov.w   #0xFFFF, R4
19
20 FIM:
21     jmp     $      ; Loop infinito
22     nop
```

Exemplo 2 (0 ponto)

Neste exemplo, o objetivo é criar um programa que receba valores X e Y em R4 e em R5 e obtenha o produto dos dois usando um loop. Seu programa deve começar movendo um byte para R4 e outro para R5 que deverão ser interpretados como números sem sinal, salvar o produto em R6 e então ficar preso em um loop infinito. A ideia é somar Y vezes o valor de X. Note que temos que restringir os valores de X e Y a números que podem ser representados com apenas 1 byte para que o resultado da multiplicação não exceda 2 bytes.

A Figura 2 mostra um fluxograma para este exercício. Crie um projeto e implemente um código em assembly a partir do fluxograma.

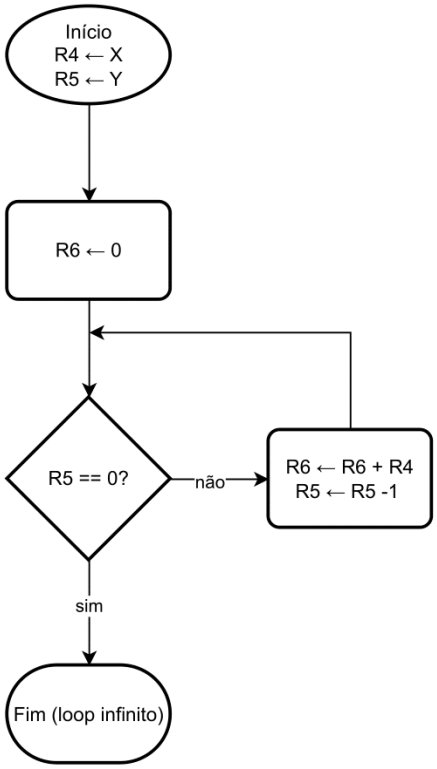


Figura 2 - Fluxograma para o Exemplo 2

Visto 1 (2 pontos)

Crie a sub-rotina **reduceSum8** que retorna o somatório de todos os bytes de um vetor. Suponha que os vetores usados serão sempre tais que resultado sempre caberá em um registro de 8 bits. A sub-rotina deve receber em R12 o endereço do vetor de bytes e em R13 a quantidade de elementos do vetor. A sub-rotina deve retornar em R12 a soma dos elementos do vetor.

Sugestão: A Figura 3 mostra um fluxograma com o programa principal chamando a sub-rotina e a sub-rotina em si à direita. Note que R4 é usado temporariamente dentro da sub-rotina para fazer a soma e, ao final, o seu valor é passado para R12 (pois o exercício pede para retornar a soma em R12). O PUSH e o POP são usados para que o valor de R4 antes da chamada da sub-rotina não seja perdido (isto não é necessário neste programa em particular, mas é uma boa prática). Note que R12 é usado como ponteiro para o elemento do vetor que vamos somar (e por isso é incrementado a cada iteração) e R13 conta quantos elementos ainda falta somar.

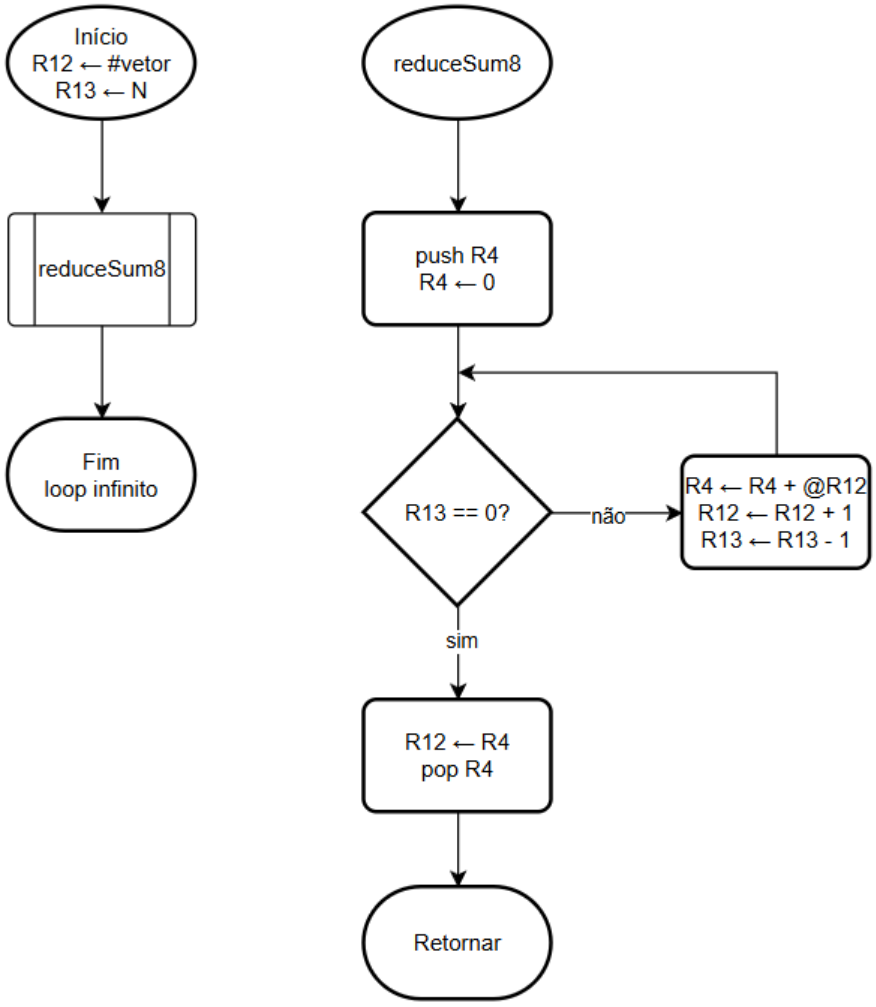


Figura 3 – Fluxograma sugerido para o Exercício 1

Visto 2 (2 pontos)

Escreva a sub-rotina **EXTREMOS**, que recebe em R12 o endereço de um vetor de words (que devem ser interpretadas como números com sinal) e em R13 a quantidade de elementos do vetor. A sub-rotina deve retornar em R12 o menor elemento do vetor e em R13 o maior elemento do vetor.

Problema Final do Módulo 1 (6 pontos)

Apresente a sub-rotina **MAT_TRANSP**, que recebe em R12 um ponteiro para uma matriz de words, em R13 o número de linhas da matriz e em R14 o número de colunas. A rotina deve escrever a transposta da matriz a partir da primeira posição de memória após o fim da matriz original. As matrizes devem estar escritas na memória linha por linha.

Por exemplo, considere a matriz de entrada

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

Neste caso, a memória RAM deve ser inicializada como indicado na Tabela 1. **Após a execução da sub-rotina**, os endereços de memória mostrados na Tabela 1 devem continuar armazenando os mesmos valores e os endereços seguintes devem armazenar a matriz M^T, como indicado na Tabela 2. Note que as matrizes são salvas na memória da mesma forma que vetores, apenas interpretamos que essas sequências de dados representam matrizes.

Tabela 1 - Memória do MSP430 contendo a matriz de entrada

Endereço	0x2400	0x2402	0x2404	0x2406	0x2408	0x240A
Valor	1	2	3	4	5	6

Tabela 2 - Memória do MSP430 contendo a matriz transposta

Endereço	0x240C	0x240E	0x2410	0x2412	0x2414	0x2416
Valor	1	4	2	5	3	6

O programa deve ter a seguinte estrutura:

```
    .cdecls "msp430.h"
    .global main
    .text
main:
    mov.w   #(WDTPW|WDTHOLD), &WDTCTL
    mov     #matriz, R12 ; Ponteiro para a matriz de entrada
    mov     #2, R13      ; Número de linhas da matriz
    mov     #3, R14      ; Número de colunas da matriz
    call    #MAT_TRANSP  ; Chamar sub-rotina
    jmp     $            ; Loop infinito
    nop
```

```
MAT_TRANSP:
    ; Seu código aqui

; Especificar a matriz de entrada na seção de dados
    .data
matriz: .word 1, 2, 3, 4, 5, 6
```

Note que a sub-rotina deve funcionar para **matrizes de qualquer tamanho** e que o programa tem que chamar a sub-rotina MAT_TRANSP, retornar para o bloco principal e então ficar preso no loop infinito.